Assignment Instructions

The assignment requires students to:

1. Study the logger factory pattern in script.js
2. Implement a counter factory with mutable state in assignment.js
3. Create two isolated counter instances (userCounter and dataCounter)
4. Set up button event listeners that increment and display counter values
5. Verify that the two counters operate independently (closure isolation)

Assignment: JavaScript Closures & Scoped Functions - Counter Factory

Objective

In this assignment, you will explore and demonstrate your understanding of **closures**, **nested functions**, and **lexical scoping** in JavaScript by implementing a counter factory pattern. This builds on the logger factory concept from script.js but uses mutable state instead of immutable context prefixes.

Background

A **closure** is created when an inner function retains access to the scope and variables of its outer (enclosing) function, even after that outer function has finished executing. The **counter factory pattern** uses this mechanism to create objects with persistent private state. Unlike the logger factory which captures immutable context, this pattern captures mutable variables that can be incremented and changed over time. **Lexical scoping** determines how variable lookup works — inner functions can "look out" to their outer scope to find and modify the counter variable.

Task

1. **Read and understand** the script.js file in the project, which demonstrates the logger factory pattern using loggerFactory(contextPrefix).
2. **Implement a counter factory** in assignment.js that follows a similar pattern but tracks mutable state:
    - A factory function counterFactory(initialValue) that returns a closure
    - The closure function increment(incrementBy) that updates and returns the captured counter value
    - Two distinct counter instances: userCounter and dataCounter, each bound to different initial values
1. **Set up event listeners** so that clicking two different buttons triggers the respective counter functions, updating the displayed output.
2. **Display output** in the #output element, showing the current counter values after each increment.

Expected Implementation Structure

Your assignment.js should contain code similar to the following pattern:

```
// Counter factory function
const counterFactory = (initialValue) => {
    // Private counter variable in the outer scope
    let counter = initialValue;

    // Inner function that forms the closure
    const increment = (incrementBy) => {
        counter += incrementBy;
        return counter;
    };

    // Return the specialized function
    return increment;
};

// Create two distinct counter instances
const userCounter = counterFactory(0);
const dataCounter = counterFactory(100);

// Event listeners
document.getElementById('button1').addEventListener('click', () => {
    userCounter(1);
    userCounter(5);
    // Update DOM with new values
});

document.getElementById('button2').addEventListener('click', () => {
    dataCounter(10);
    dataCounter(20);
    // Update DOM with new values
});
```

Key Concepts to Demonstrate

- **Closure with mutable state**: The inner function retains access to and can modify the counter variable
- **Function Factory Pattern**: One factory creates multiple isolated counter instances
- **Lexical Scoping**: Variable resolution and modification based on physical location in the code
- **State persistence**: Each counter maintains its own private state across multiple calls

- **Isolation**: userCounter and dataCounter maintain separate private counters

Submission

Ensure all three files (assignment.html, assignment.js, and the provided style.css) are in the same directory. Open assignment.html in a web browser and verify that:

- Clicking Button 1 increments the user counter and displays the updated values
- Clicking Button 2 increments the data counter and displays the updated values
- The two counters operate independently — incrementing one does not affect the other
- Each click returns the current counter value (closure retains state)

Zip these up and submit them here.

Submission

Click to add text

Word count: 0

Drag files here

Maximum size: 512 MB

or

Upload a file **Upload from cloud**

Save and Close **Submit**